

BANCO DE DADOS

com Microsoft SQL Server

AULA 02

DDL — Linguagem de Definição de Dados

CREATE DATABASE

CREATE TABLE

ALTER TABLE

DROP/TRUNCATE

Constraints

Integridade Ref.



Objetivos da Aula 02



Dominar CREATE DATABASE com opções avançadas



Implementar: NOT NULL, UNIQUE, CHECK, DEFAULT



Criar tabelas com estrutura completa de constraints



Configurar integridade referencial FK completa



Modificar tabelas com ALTER TABLE



Diferença entre TRUNCATE e DELETE



Excluir objetos com segurança usando DROP



Praticar no SISTEMA_VENDAS com scripts reais



Revisão — Aula 01

? O que é Banco de Dados?

✓ Coleção organizada de dados gerenciados por um SGBD

? O que é um SGBD?

✓ Software que gerencia armazenamento, segurança e transações

? O que é Chave Primária (PK)?

✓ Identificador único de cada registro — não nulo, não repetido

? O que é Chave Estrangeira (FK)?

✓ Campo que referencia PK de outra tabela — cria relacionamento

? O que é IDENTITY(1,1)?

✓ Gerador automático de PK: início=1, incremento=1

DDL — O que é e Comandos Principais

DDL (Data Definition Language): subconjunto SQL para DEFINIR, MODIFICAR e REMOVER estruturas. Cria o esqueleto que suporta os dados.



CREATE

Cria objetos: banco, tabela, view, index, procedure



ALTER

Modifica estrutura de objetos existentes



DROP

Remove permanentemente objetos do banco



TRUNCATE

Remove todos dados rapidamente (mais rápido que DELETE)



RENAME

Renomeia objetos (sp_rename no SQL Server)



COMMENT

Adiciona comentários e descrição a objetos



CREATE DATABASE — Opções Completas

```
-- Sintaxe simplificada
CREATE DATABASE SISTEMA_VENDAS; GO

-- Sintaxe completa com configurações de arquivo
CREATE DATABASE SISTEMA_VENDAS
ON PRIMARY (
    NAME            = 'SISTEMA_VENDAS',
    FILENAME        = 'C:\SQLData\SV.mdf',
    SIZE            = 10MB,
    MAXSIZE         = 500MB,
    FILEGROWTH      = 10%
)
LOG ON (
    NAME            = 'SISTEMA_VENDAS_log',
    FILENAME        = 'C:\SQLData\SV.ldf',
    SIZE            = 5MB,
    MAXSIZE         = 100MB,
    FILEGROWTH      = 10%
)
COLLATE Latin1_General_CI_AI; -- CI=Case Insensitive, AI=Accent Insensitive
GO

-- Verificar banco criado
SELECT name, state_desc, recovery_model_desc
FROM    sys.databases WHERE name='SISTEMA_VENDAS';
```

Constraints — Regras de Integridade

PRIMARY KEY

```
CONSTRAINT PK_CLIENTES PRIMARY KEY (id_cliente)
```

Identifica unicamente cada linha. NOT NULL+UNIQUE. Máx 1 por tabela.

NOT NULL

```
nome VARCHAR(150) NOT NULL
```

Campo obrigatório. Tentativa de NULL gera erro imediato.

CHECK

```
CONSTRAINT CK_PROD_PRECO CHECK (preco > 0)
```

Valida expressão lógica. Ex: preço>0, status IN lista.

FOREIGN KEY

```
CONSTRAINT FK_PROD_CAT FOREIGN KEY (id_cat) REFERENCES  
CATEGORIAS(id_categoria)
```

Relacionamento entre tabelas. Garante que o valor existe na tabela pai.

UNIQUE

```
CONSTRAINT UQ_CLI_EMAIL UNIQUE (email)
```

Sem valores repetidos. Permite NULL (diferente de PK).

DEFAULT

```
ativo BIT NOT NULL DEFAULT 1
```

Valor padrão quando não informado. DEFAULT GETDATE() para data.



ALTER TABLE — Modificando Estruturas

```
USE SISTEMA_VENDAS; GO
```

```
-- 1. ADICIONAR coluna
```

```
ALTER TABLE CLIENTES ADD cpf CHAR(11) NULL; GO
```

```
-- 2. MODIFICAR tipo/tamanho
```

```
ALTER TABLE CLIENTES ALTER COLUMN email VARCHAR(300) NULL; GO
```

```
-- 3. REMOVER coluna
```

```
ALTER TABLE CLIENTES DROP COLUMN telefone; GO
```

```
-- 4. ADICIONAR constraint após criação
```

```
ALTER TABLE CLIENTES
```

```
ADD CONSTRAINT UQ_CLI_CPF UNIQUE (cpf); GO
```

```
-- 5. REMOVER constraint
```

```
ALTER TABLE CLIENTES DROP CONSTRAINT UQ_CLI_CPF; GO
```

```
-- 6. ADICIONAR coluna com DEFAULT em tabela com dados
```

```
ADD data_atualizacao DATETIME2 NULL
```

```
CONSTRAINT DF_CLI_DTATU DEFAULT GETDATE(); GO
```

```
ALTER TABLE CLIENTES
```

```
-- 7. RENOMEAR coluna (usa procedure sp_rename)
```

```
EXEC sp_rename 'CLIENTES.celular','telefone','COLUMN'; GO
```

DROP vs TRUNCATE vs DELETE — Diferenças Críticas

Característica	DROP TABLE	TRUNCATE TABLE	DELETE FROM
O que faz?	Remove a TABELA	Remove todos DADOS	Remove dados (com filtro)
Estrutura mantida?	❌ Destruída	✅ Sim	✅ Sim
IDENTITY reset?	N/A	✅ Sim	❌ Não
Pode usar WHERE?	❌ Não	❌ Não	✅ Sim
Velocidade	Imediato	⚡ Muito rápida	🐢 Mais lenta
ROLLBACK possível?	❌ Não (DDL)	❌ Não (DDL)	✅ Sim (DML)
Triggers disparam?	❌ Não	❌ Não	✅ Sim

Integridade Referencial — ON DELETE e ON UPDATE



NO ACTION

Padrão. Lança ERRO se tentar excluir/alterar o pai com filhos existentes.

Ex: Não deixa excluir **CATEGORIA** se tiver **PRODUTO** vinculado



CASCADE

A ação no pai se propaga para filhos automaticamente.

Ex: Excluiu **PEDIDO** → exclui todos os **ITENS_PEDIDO** automaticamente



SET NULL

FK nos filhos vira NULL quando o pai é excluído/alterado.

Ex: Excluiu **CATEGORIA** → **PRODUTO** fica com **id_categoria = NULL**



SET DEFAULT

FK nos filhos recebe valor DEFAULT quando o pai muda.

Ex: Excluiu **FUNCIONARIO** → **PEDIDO.id_funcionario = valor default**



CHECK Constraints — Validações de Negócio

```
USE SISTEMA_VENDAS; GO

-- CHECK: preço deve ser maior que zero
ALTER TABLE PRODUTOS
ADD CONSTRAINT CK_PROD_PRECO CHECK (preco > 0); GO

-- CHECK: estoque não pode ser negativo
ALTER TABLE PRODUTOS
ADD CONSTRAINT CK_PROD_ESTOQUE CHECK (estoque >= 0); GO

-- CHECK: status = lista de valores permitidos
ALTER TABLE PEDIDOS
ADD CONSTRAINT CK_PED_STATUS
    CHECK (status IN ('PENDENTE','APROVADO','ENVIADO','ENTREGUE','CANCELADO')); GO

-- CHECK: salário dentro de faixa razoável
ALTER TABLE FUNCIONARIOS
ADD CONSTRAINT CK_FUNC_SALARIO
    CHECK (salario BETWEEN 1412.00 AND 50000.00); GO

-- Testar: INSERT inválido deve gerar erro!
INSERT INTO PRODUTOS (nome_produto,preco,estoque)
VALUES ('Produto Teste', -10.00, 5); -- ERRO: CHECK violado!

-- Listar CHECKs definidos
SELECT tc.TABLE_NAME, tc.CONSTRAINT_NAME, cc.CHECK_CLAUSE
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS tc
JOIN INFORMATION_SCHEMA.CHECK_CONSTRAINTS cc ON tc.CONSTRAINT_NAME=cc.CONSTRAINT_NAME;
```



Script DDL Completo — SISTEMA_VENDAS v2.0

```
-- Drop na ordem inversa das FKs
IF OBJECT_ID('ITENS_PEDIDO','U') IS NOT NULL DROP TABLE ITENS_PEDIDO;
IF OBJECT_ID('PEDIDOS','U')      IS NOT NULL DROP TABLE PEDIDOS;
IF OBJECT_ID('PRODUTOS','U')     IS NOT NULL DROP TABLE PRODUTOS;
IF OBJECT_ID('FUNCIONARIOS','U') IS NOT NULL DROP TABLE FUNCIONARIOS;
IF OBJECT_ID('CLIENTES','U')     IS NOT NULL DROP TABLE CLIENTES;
IF OBJECT_ID('CATEGORIAS','U')   IS NOT NULL DROP TABLE CATEGORIAS;
GO

CREATE TABLE CATEGORIAS (id_categoria INT IDENTITY(1,1) NOT NULL,
    nome_categoria VARCHAR(100) NOT NULL, descricao VARCHAR(500) NULL,
    ativo BIT NOT NULL DEFAULT 1,
    CONSTRAINT PK_CATEGORIAS PRIMARY KEY (id_categoria),
    CONSTRAINT UQ_CAT_NOME    UNIQUE (nome_categoria)); GO

CREATE TABLE CLIENTES (id_cliente INT IDENTITY(1,1) NOT NULL,
    nome VARCHAR(150) NOT NULL, email VARCHAR(200) NULL,
    telefone CHAR(11) NULL, cidade VARCHAR(100) NULL, estado CHAR(2) NULL,
    data_cadastro DATETIME2 NOT NULL DEFAULT GETDATE(),
    CONSTRAINT PK_CLIENTES PRIMARY KEY (id_cliente)); GO

CREATE TABLE PRODUTOS (id_produto INT IDENTITY(1,1) NOT NULL,
    nome_produto VARCHAR(200) NOT NULL, preco DECIMAL(10,2) NOT NULL,
    estoque INT NOT NULL DEFAULT 0, id_categoria INT NULL,
    CONSTRAINT PK_PRODUTOS  PRIMARY KEY (id_produto),
    CONSTRAINT CK_PROD_PRC  CHECK (preco > 0),
    CONSTRAINT FK_PROD_CAT  FOREIGN KEY (id_categoria)
        REFERENCES CATEGORIAS(id_categoria) ON DELETE SET NULL); GO
```



Exercícios Propostos — Aula 02

01

Básico

Adicione à tabela CLIENTES: colunas cep CHAR(8) NULL e complemento VARCHAR(100) NULL. Adicione CHECK garantindo que estado tenha 2 caracteres (LEN(estado)=2).

02

Básico

Crie FORNECEDORES (id PK IDENTITY, nome NOT NULL, cnpj CHAR(14) UNIQUE, email VARCHAR(200), ativo BIT DEFAULT 1). Adicione id_fornecedor FK em PRODUTOS.

03

Médio

Adicione CHECK em PEDIDOS: data_pedido deve ser <= GETDATE() (não aceita datas futuras). Teste inserindo pedido com data futura e observe a mensagem de erro.

04

Médio

Crie script de 'reset' do SISTEMA_VENDAS: verifique cada tabela com OBJECT_ID e, se existir, remova. Garanta a ordem correta respeitando FKs.

05

Avançado

Adicione coluna calculada subtotal = quantidade * preco_unitario PERSISTED na tabela ITENS_PEDIDO. Verifique com sys.columns que is_computed=1 e is_persisted=1.



Respostas Comentadas — Ex 01, 03 e 05

```
-- EX 01: Adicionar colunas e CHECK
ALTER TABLE CLIENTES ADD cep CHAR(8) NULL;
ALTER TABLE CLIENTES ADD complemento VARCHAR(100) NULL;
ALTER TABLE CLIENTES ADD CONSTRAINT CK_CLI_ESTADO
    CHECK (LEN(estado) = 2); GO

-- EX 03: CHECK data pedido não futura
ALTER TABLE PEDIDOS ADD CONSTRAINT CK_PED_DATA
    CHECK (data_pedido <= GETDATE()); GO

-- Teste: deve gerar erro de CHECK constraint
INSERT INTO PEDIDOS (id_cliente, data_pedido, status)
VALUES (1, '2099-01-01', 'PENDENTE'); -- ERRO esperado!

-- EX 05: Coluna calculada PERSISTIDA
ALTER TABLE ITENS_PEDIDO
ADD subtotal AS (quantidade * preco_unitario) PERSISTED; GO

-- Verificar: is_computed=1, is_persisted=1
SELECT name, is_computed, is_persisted
FROM sys.columns
WHERE object_id = OBJECT_ID('ITENS_PEDIDO');
```



Desafio da Aula 02

 Contexto: A empresa quer expandir o SISTEMA_VENDAS com controle de fornecedores, endereços e histórico de preços.

Tarefas:

1. Criar tabela FORNECEDORES com PK, UNIQUE no CNPJ e campos completos
2. Adicionar FK em PRODUTOS referenciando FORNECEDORES (ON DELETE SET NULL)
3. Criar tabela ENDERECOS e adicionar FK em CLIENTES
4. Criar tabela HISTORICO_PRECOS (id, id_produto FK, preco_ant, preco_nov, data)
5. Todos os constraints com nomenclatura padronizada: PK_, FK_, CK_, UQ_, DF_
6. BÔNUS: Criar índice em CLIENTES(email) e PRODUTOS(nome_produto) para buscas rápidas



Resumo — Aula 02: DDL

CREATE



Banco, tabela e objetos com opções completas de arquivo e collation

ALTER TABLE



Adiciona/modifica/remove colunas e constraints em tabelas existentes

DROP vs TRUNCATE



DROP remove estrutura; TRUNCATE remove dados (DDL, sem rollback, IDENTITY reseta)

Constraints



PK, FK, NOT NULL, UNIQUE, CHECK, DEFAULT — guardiões da integridade

Integridade Referencial



ON DELETE/UPDATE: NO ACTION, CASCADE, SET NULL, SET DEFAULT

CHECK



Valida regras de negócio no banco: preço>0, status em lista, etc.

Boas Práticas



OBJECT_ID antes de DROP, ordem inversa nas FKs, nomenclatura padronizada

Verificação



INFORMATION_SCHEMA, sys.indexes, sys.columns para inspecionar tudo



Próxima Aula 03: Teoria dos Conjuntos no SQL



Conjuntos Matemáticos

Base teórica do modelo relacional



Pertinência e Inclusão

Representação no SQL: IN, EXISTS



União — UNION

Combinar resultados de múltiplas queries



Interseção — INTERSECT

O que há em comum entre conjuntos



Diferença — EXCEPT

O que há em um mas não no outro



Complemento

Dados fora de um conjunto específico



Tarefa: Revisar conjuntos da matemática e finalizar desafio da Aula 02.